```
alias_method :respond_to_without_attributes?, :respond_to?

def respond_to?(method)
  @@dynamic_methods ||= attribute_names +
                        attribute_names.collect { |attr| attr + "=" } +
                        attribute_names.collect { |attr| attr + "?" }
  @@dynamic_methods.include?(method.to_s) ?
                                          true :
                                          respond_to_without_attributes?(method)
end

def method_missing(method_id, *arguments)
  method_name = method_id.id2name

  if method_name =~ read_method? && @attributes.include?($1)
    return read_attribute($1)
  elsif method_name =~ write_method?
    write_attribute($1, arguments[0])
  elsif method_name =~ query_method?
    return query_attribute($1)
  else
    super
  end
end

def read_method?()  /^([a-zA-Z][-_\w]*)[^=?]*$/ end
def write_method?() /^([a-zA-Z][-_\w]*)=.*$/    end
def query_method?() /^([a-zA-Z][-_\w]*)\?$/     end

def read_attribute(attr_name)         # ...
def write_attribute(attr_name, value) #...
def query_attribute(attr_name)        # ...
```

Take a look at the initialize method: when you create an ActiveRecord::Base object, its @attributes instance variable is populated with the name of the attributes from the database. For example, if the relevant table in the database has a column named description, then @attributes will contain the string "description", among others.

Now skip down to method_missing, where those attribute names become the names of *Ghost Methods* (57). When you call a method such as description=, method_missing notices two things: first, description is the name of an attribute; and second, the name of description= matches the regular expression for write accessors. As a result, method_missing calls write_attribute("description"), which writes the value of the description in the database. A similar process happens for query accessors (that end in a question mark) and read accessors (that are just the same as attribute names).